

GLUCOEDGE: An Engineering Study of On-Device Five-Class Glucose Trend Forecasting and INT8 Quantization

Mohamed Menasy
Independent Researcher
mohamedmenasy@gmail.com

July 2026

Abstract

Published evaluations of continuous glucose monitor (CGM) forecasting largely emphasize offline predictive accuracy, while edge deployment also requires small artifacts, deterministic preprocessing, cross-runtime numerical parity, and measurements on real hardware. This paper presents GLUCOEDGE, an open-source Android prototype that maps the previous hour of 5-minute CGM readings to one of five rate-of-change classes 15 minutes ahead. A 2,909-parameter one-dimensional convolutional neural network was trained with class-weighted cross-entropy on the public 200-participant Weinstock data as prepared by GlucoBench. The resulting data frames produced 424,462 training, 55,289 validation, and 127,165 aggregate test windows. Float LiteRT conversion preserved the PyTorch classification metrics: 0.5184 accuracy and 0.5060 macro-averaged recall. Static INT8 post-training quantization reduced the model from 16.95 to 11.70 KiB (31.0%), but changed accuracy to 0.5866 while reducing macro recall to 0.4135. The apparent accuracy gain resulted from increased prediction of the 75.5%-prevalence **stable** class; recall decreased for all four minority classes, including a 62.7% relative reduction for **falling_fast**. On a Samsung Galaxy S22 Ultra, the full float classification path measured 0.334 ms mean and 0.540 ms p95 latency, compared with 0.484 ms and 0.665 ms for INT8. In this case study, the device observations and the narrow INT8 calibration range show why quantization outcomes must be measured for a given model-runtime-hardware path rather than inferred from model size or aggregate accuracy. GLUCOEDGE is a research and engineering demonstration, not a medical device or treatment system.

1 Introduction

A five-minute CGM cadence yields 12 observations per hour. The resulting dense time series supports computational estimates of a near-future value, event, or trend [1]. This paper treats those estimates as research outputs, not treatment inputs.

These targets define different learning problems. Value regression estimates a future glucose level, event prediction asks whether a glycemic event will occur, and trend classification assigns a directional rate category. Published studies also differ in prediction horizon, CGM-only versus contextual inputs, cohort, split construction, and metric. Reviews and comparison indices document this heterogeneity, while public benchmarks and cross-dataset experiments expose how strongly evaluation scope shapes the question being answered [1–4]. Headline metric values from incompatible protocols should therefore not be compared directly.

Commercial CGM trend arrows are related but not equivalent targets. They summarize recent or current glucose direction and rate using device-specific definitions [5, 6]. In contrast, GLUCOEDGE derives each label from the change between the current sample and the sample 15 minutes later. The arrow-like rate bands provide a compact research representation, but the future-derived label neither reproduces a manufacturer’s algorithm nor supplies a dosing rule.

Deployment adds another evaluation layer. A model that performs well in a desktop framework may change numerically after conversion, require different input handling in the client, or fail to gain latency from lower precision on a particular device. Class imbalance makes conversion errors easy to conceal: accuracy can increase as predictions concentrate on a majority class even while minority-class recall declines [7].

This work provides a reproducible offline replay path from a public CGM data frame through training and conversion to measured Android execution. It does not claim clinical forecasting performance or treatment utility. Its contributions are:

- a segment-aware construction of one-hour CGM windows and future-derived, five-class 15-minute trend labels;
- an empirical test of whether the two rare “fast” classes have sufficient support to retain a five-class formulation;
- matched evaluation of PyTorch, float LiteRT, and

static INT8 LiteRT artifacts on the same aggregate test frame;

- cross-runtime golden-vector tests and physical-device latency observations for an `Android CompiledModel` implementation; and
- a case-study failure analysis showing that a 31.0% size reduction coincided with a 9.25-point decrease in macro recall and no latency advantage on the measured device.

All source code, tests, synthetic replay data, and shipped model artifacts are available in the project repository.¹

2 Related Work

2.1 Forecasting tasks, horizons, and inputs

CGM forecasting has expanded from autoregressive value prediction using sensor history [8] to recurrent and attention models trained with CGM and contextual variables [9]. Recurrent models have also been used to estimate predictive variance rather than only a point value [10], while convolutional-recurrent models fuse CGM, insulin, and meal sequences before predicting 30- or 60-minute values [11]. Architecture and input modality change together across these studies, so their results do not isolate the effect of model family alone. Cross-dataset evaluation asks a different question: whether a trained model generalizes when the source dataset, cohort, and collection process change [4].

Table 1 records these design differences without placing incompatible metric values side by side. It also separates offline evaluation from reported device execution, a distinction that predictive scores alone do not capture.

2.2 Benchmarks and evaluation protocols

Evaluation scope determines the generalization claim. Subject-dependent splits estimate performance for people represented during training, held-out-participant splits test transfer to unseen people within a dataset, and cross-dataset evaluation adds changes in cohort and data pipeline. GlucoBench standardizes preprocessing, splits, and baselines over five public datasets to support comparisons within a declared protocol [1]; experiments across four datasets directly test changes in data source [4].

GLUCOEDGE uses GlucoBench’s largest type-1 diabetes cohort, referred to as Weinstock. Its source study collected blinded CGM from older adults with long-standing type-1 diabetes [12].

Time-series splitting also requires care because nearby and overlapping windows are dependent. The validity of cross-validation for autoregressive prediction depends on the residual structure rather than a generic random-split rule [13], and leakage can produce answers to a less demanding problem than the one a study claims to test [14]. Even with a fixed split, training and implementation choices contribute benchmark variance [15]. Reporting participant, temporal, and dataset boundaries is therefore part of the result, not merely a preprocessing detail.

2.3 Trend representations and class imbalance

Commercial trend arrows condense retrospective or current movement into device-specific categories whose rate bands and look-back intervals differ [5, 6]. A future label instead asks whether the trajectory after the input window falls, remains stable, or rises. GLUCOEDGE borrows rate bands as a research representation but computes them from the sample 15 minutes after the window; it neither emulates a commercial arrow algorithm nor supports treatment decisions.

In the Weinstock windows used here, most labels are stable and faster changes are relatively rare. Overall accuracy therefore weights the stable class in proportion to its prevalence. Macro-averaged recall, equivalent to balanced accuracy for this single-label setting, gives each class equal weight and makes minority-class degradation visible [7]. Macro recall still does not establish event-level or clinical performance.

2.4 Edge inference and quantization

One-dimensional CNNs provide local temporal feature extraction through compact feed-forward interfaces [16].

Mobile inference systems can reduce neural-network execution cost through compression and runtime-specific scheduling [17], but latency is a property of the model, runtime, and hardware together. Li et al. reported Android timing for a glucose forecasting model [11]; broader smartphone and tiny-device benchmarks likewise measure defined workloads on specified platforms rather than treating artifact size as a speed proxy [18, 19].

Integer quantization approximates floating-point tensors with discrete values parameterized by a scale and zero point [20]. Static post-training quantization estimates activation ranges from calibration inputs before conversion [21, 22]. Low-bit studies show that clipping, range estimation, and rounding choices can determine the resulting error [23, 24]; specialized methods have also been proposed to improve post-training accuracy when calibration sets are small [25]. These findings motivate checking converted outputs and class-wise metrics before drawing conclusions from reduced model size.

¹<https://github.com/mohamedmenasy/GlucoEdge>

Table 1: Representative CGM forecasting studies and evaluation scope. Metrics are not compared across rows because datasets and protocols differ.

Study	Target	Horizon	Inputs	Evaluation scope	Device evidence
Reifman et al. [8]	Value	30 min	CGM	9 participants	None reported
Mirshakarian et al. [9]	Value	30/60 min	CGM + context	OhioT1DM/synthetic	None reported
Martinsson et al. [10]	Value + variance	30/60 min	CGM	OhioT1DM	None reported
Li et al. [11]	Value	30/60 min	CGM, insulin, meals	10 real + 10 simulated	Android timing
Glucobench [1]	Value	Multiple	Dataset-dependent	Five public datasets	None reported
Ghimire et al. [4]	Value	30/60 min	CGM	Four datasets	None reported
GLUCOEDGE (this work)	Trend class	15 min	CGM	Weinstock aggregate test	Android replay timing

GLUCOEDGE connects public preprocessing with conversion parity, class-wise evaluation, and offline replay on one Android device; it does not establish live-sensor, clinical, or cross-hardware performance.

3 Problem Formulation

Let g_t be the CGM value at time t , in mg/dL, with observations every five minutes. The input is

$$\mathbf{x}_t = [g_{t-55}, g_{t-50}, \dots, g_t] \in \mathbb{R}^{12}. \quad (1)$$

The future rate of change is computed from a point outside the input window:

$$r_t = \frac{g_{t+15} - g_t}{15} \quad [\text{mg/dL/min}]. \quad (2)$$

The class y_t is assigned by

$$y_t = \begin{cases} \text{falling_fast}, & r_t \leq -2, \\ \text{falling}, & -2 < r_t \leq -1, \\ \text{stable}, & -1 < r_t < 1, \\ \text{rising}, & 1 \leq r_t < 2, \\ \text{rising_fast}, & r_t \geq 2. \end{cases} \quad (3)$$

The 60-minute history and 15-minute horizon are fixed engineering choices for this prototype, not optimized clinical values. The five-class target is a simplified future-rate abstraction based on two samples rather than a complete description of the intervening trajectory. The boundaries use rate bands commonly represented by CGM trend arrows, while collapsing device-specific arrow taxonomies into five classes [5, 6]. No threshold was optimized on the test labels.

4 Data and Methods

4.1 Dataset and preprocessing

The experiments use the Weinstock data distributed through GlucoBench. The source cohort comprised older adults with long-standing type-1 diabetes and blinded CGM collection [12]. The GlucoBench release reports 200 participants and a five-minute sampling interval [1].

Table 2: Window distribution after GlucoBench formatting. Percentages are within each split.

Class	Train	Aggregate test
falling_fast	13,260 (3.12%)	3,692 (2.90%)
falling	40,122 (9.45%)	11,387 (8.95%)
stable	314,257 (74.04%)	95,998 (75.49%)
rising	39,428 (9.29%)	11,012 (8.66%)
rising_fast	17,395 (4.10%)	5,076 (3.99%)
Total	424,462	127,165

Its formatter retains glucose in raw mg/dL, interpolates within gaps of at most 45 minutes, drops short segments, and tags each contiguous result with an `id_segment`. Because interpolation precedes windowing, generated values may occur among the inputs or at samples used to form targets. The configured split holds out 10% of participants for an out-of-distribution component and forms temporal validation and test components from the remaining participants. The formatter’s reported test frame concatenates these components; the present work reports that aggregate rather than separate in- and out-of-distribution metrics.

Windows are generated independently within each (`id`, `id_segment`) group and ordered by time. A window is emitted only when all 12 input points and the three following horizon steps fit in the same segment. The stride is one reading, matching an application that updates every five minutes. This rule prevents a sensor gap or participant boundary from being silently interpreted as a continuous trajectory. Adjacent stride-one windows overlap heavily, so the window totals are dependent samples rather than independent event counts.

Table 2 shows the resulting class imbalance. Both fast classes exceed 13,000 training windows. A preliminary 20-epoch run obtained test recalls of 0.5333 and 0.5380 for `falling_fast` and `rising_fast`, respectively, so a predeclared collapse trigger of fewer than 200 training examples or recall below 0.30 did not fire. The deployed artifact therefore retains all five classes. Because test recall informed this five-versus-three decision, it was an exploratory, test-informed development choice rather than unbiased validation evidence.

4.2 Model and training

The **TrendCNN** accepts tensors of shape $(B, 1, 12)$. It applies a one-dimensional convolution from 1 to 24 channels, a second convolution from 24 to 36 channels, ReLU after each layer, adaptive average pooling over time, and a 36-to-5 linear head. Both convolutions use kernel size 3, stride 1, and padding 1. The network contains 2,909 trainable parameters.

Training uses inverse-frequency class-weighted cross-entropy, batch size 64, and Adam with learning rate 10^{-3} [26]. The artifact used for conversion was trained for 20 epochs with PyTorch seed 0 [27]. Validation loss was observed after every epoch; there was no early stopping or test-time selection. Classification reports were computed with scikit-learn [28]. Because the test distribution is dominated by **stable**, macro-averaged recall is the primary aggregate metric. For single-label multiclass prediction it is the mean of the five per-class recalls and is equivalent to balanced accuracy [7].

5 Conversion and Deployment

The seed-0 checkpoint was converted with LiteRT Torch. Float conversion uses a fixed $(1, 1, 12)$ input signature. INT8 conversion uses PyTorch 2 Export (PT2E) with a static symmetric, per-channel configuration and the first 200 windows from the validation data as calibration inputs. For an input scale s , zero point z , and signed INT8 limits, Android reproduces the conversion benchmark’s quantization rule:

$$q = \text{clip}(\text{round}(x/s) + z, -128, 127), \quad \hat{x} = s(q - z). \quad (4)$$

Quantization parameters are read from the model meta-data rather than hardcoded.

The Kotlin/Jetpack Compose application replays a bundled synthetic trace. A 12-reading ring buffer resets after a cadence gap, mirroring training-time segment boundaries. Every complete window is classified locally; the merged Android manifest is tested to ensure it does not request the **INTERNET** permission. The float artifact is the default and INT8 can be selected for comparison.

Real devices use LiteRT 2.1.0 `CompiledModel` on CPU [29]. On the tested Apple-Silicon-hosted Android emulator, native model creation terminated with an illegal-instruction fault during an ARM SVE feature probe. Because this fault cannot be caught by Kotlin, emulators are selected before runtime initialization and use the classic interpreter with **XNNPACK** disabled. This workaround is an environment-specific engineering result, not a general LiteRT performance claim.

6 Experimental Protocol

The aggregate test frame combines temporal and held-out-participant components. As detailed in Section 9,

Table 3: Aggregate test metrics. PyTorch and float LiteRT have identical reported values.

Model	Accuracy	Macro recall
Constant stable	0.7549	0.2000
PyTorch checkpoint	0.5184	0.5060
Float LiteRT	0.5184	0.5060
INT8 LiteRT	0.5866	0.4135

context-prefix rows were not masked as target starts in this implementation; the resulting metrics document the shipped artifact but are not a leakage-free estimate of participant-level generalization.

The same 127,165 aggregate test windows and labels were passed through the PyTorch checkpoint, float LiteRT model, and INT8 LiteRT model. Accuracy, per-class recall, and macro recall were computed from each prediction set. A constant-**stable** classifier is included to expose the effect of class prevalence.

File size is the exact serialized artifact size. Device latency is measured inside the application’s `classify` method with `System.nanoTime`; it includes buffer writes, input quantization when needed, model execution, output dequantization, and softmax. Measurements were observed during replay on a Samsung Galaxy S22 Ultra (SM-S908E, Android 16). The float statistics use the latest 100 inferences, while the recorded INT8 statistics contain 21 inferences after a model switch. These are application- path observations, not a controlled microbenchmark.

Cross-runtime parity was tested with 20 synthetic golden windows, four selected from each float-predicted class and including values above the INT8 input range. The test also verifies the SHA-256 hash of each bundled artifact. Float logits must match the Python output within 10^{-5} ; dequantized INT8 outputs are required to match bit-for-bit because both sides dequantize identical integer outputs with the same float32 operation.

7 Results

7.1 Predictive performance

Table 3 shows that float LiteRT preserved the checkpoint’s reported classification metrics. The constant majority-class baseline reaches 0.7549 accuracy, higher than either learned model, but its macro recall is only 0.2000. This contrast is why accuracy alone is inappropriate for this task.

INT8 accuracy increased by 6.82 percentage points relative to float, while macro recall decreased by 9.25 points. Table 4 localizes the change: **stable** is the only class whose recall improved. Every minority class deteriorated. The largest change was **falling_fast**, whose recall fell from 0.5119 to 0.1907, a 62.7% relative reduction.

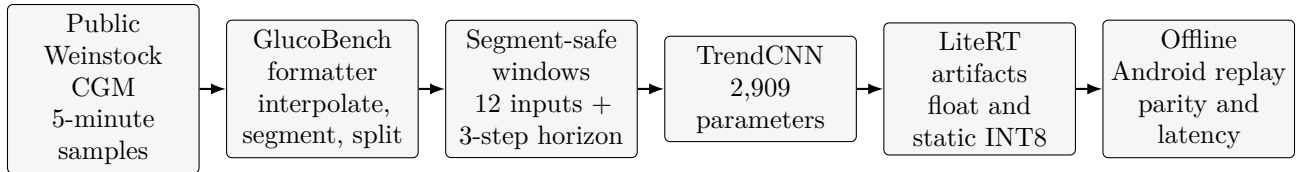


Figure 1: End-to-end GLUCOEDGE workflow. Patient data are used locally for training and evaluation; only model files and a synthetic replay trace are committed to the Android application.

Table 4: Per-class recall before and after static INT8 quantization.

Class	Float	INT8	Change
falling_fast	0.5119	0.1907	−0.3212
falling	0.5484	0.4306	−0.1178
stable	0.5225	0.6529	+0.1304
rising	0.4761	0.3845	−0.0916
rising_fast	0.4712	0.4090	−0.0622

Table 5: Artifact and physical-device observations. Latency covers the full classification path; sample counts differ by model.

Measurement	Float	INT8
Artifact size (KiB)	16.95	11.70
Mean latency (ms)	0.334 ($n = 100$)	0.484 ($n = 21$)
p95 latency (ms)	0.540	0.665
Golden vectors passed	20/20	20/20

7.2 Size, latency, and parity

Table 5 summarizes deployment measurements. INT8 reduced the artifact by 5,376 bytes, or 31.0%, but did not reduce observed on-device latency. The unequal sample counts do not support a formal latency comparison; they do rule out claiming an observed INT8 speedup in this run.

The calibration sample provides one plausible mechanism for part of the predictive degradation. Its learned input ceiling was approximately 240 mg/dL, whereas the test data reached 401 mg/dL. Approximately 28% of test windows therefore contained at least one value that saturated at the INT8 boundary. Correct clipping prevents integer overflow but cannot recover information outside the calibrated range.

All three Android instrumentation tests passed on the physical device. The `CompiledModel` path loaded both artifact hashes, matched float logits within 10^{-5} on all 20 windows, and matched INT8 dequantized outputs exactly. Thus the parity result excludes divergent Kotlin quantization arithmetic as an explanation for the measured INT8 behavior.

8 Discussion

The central result is not that quantization is uniformly harmful, but that its trade-off depends on the model, calibration set, runtime, and metric. Integer arithmetic often reduces memory and can improve latency for compute-heavy networks [20]; here the network is only 2,909 parameters, so fixed invocation and data-movement costs may dominate the measured path. Mobile acceleration and device benchmarks likewise make performance specific to the model, runtime, and hardware [17–19]. The observed artifact-size benefit was a 5.25-KiB file reduction.

The accuracy increase is an instance of the class-imbalance problem that macro recall is designed to reveal. A shift toward **stable** correctly labels more examples because three quarters of the test windows are stable, but it simultaneously reduces sensitivity to every directional class. Selecting INT8 from accuracy alone would therefore reverse the model choice implied by the task’s stated objective. GLUCOEDGE ships float by default for this reason.

The saturation analysis shows why a nominal calibration-sample count was insufficient in this conversion. Official guidance describes a few hundred randomly selected samples as typical for range calibration [22]; here, the first 200 validation windows were non-random and did not span the observed test-input range. Prior PTQ work identifies clipping and range estimation as accuracy concerns [23], treats rounding as a separate optimization problem [24], and develops methods for limited calibration data [25]. Stratified calibration over glucose ranges and trend classes, a larger calibration set, percentile-based range selection, quantization-aware training, or float16/dynamic-range alternatives [30] should be compared before rejecting quantization generally.

Finally, numerical parity tests are necessary but not sufficient. They verify that Python and Android execute the same artifacts and preprocessing, excluding divergent client arithmetic, but they do not isolate calibration from every other quantization effect. They also do not validate the labels, the cohort split, clinical safety, calibration of probabilities, or usefulness to a person with diabetes.

9 Limitations

This study has several limitations that constrain interpretation.

1. The deployable checkpoint is a single seed; training and implementation choices can contribute benchmark variance [15]. No across-seed variability or confidence intervals are reported. The study omits probability calibration, event-level evaluation, and input-dependent temporal baselines beyond the constant-`stable` comparator.
2. Stride-one windows are overlapping and statistically dependent, so their totals are not independent event counts [13]. Because GlucoBench interpolates before window construction, generated values may occur in model inputs or in samples used to form targets; results are not stratified by interpolation status.
3. GlucoBench adds prefix context to validation and in-distribution test frames so sequence models can form initial histories. The current GLUCOEDGE dataset builder treats all prefix rows as eligible window starts rather than masking context-only targets. Some aggregate validation/test windows can therefore reuse temporal context from an adjacent split. This context-prefix leakage risk can make evaluation answer a less demanding question [14]. The reported figures document the built engineering artifact, but they are not a leakage-free estimate of participant-level generalization. A corrected evaluation must mask context-only targets and report in- and out-of-distribution components separately.
4. The aggregate test frame mixes temporal hold-outs with participants held out by GlucoBench. The source cohort comprised older adults with long-standing type-1 diabetes and blinded CGM collection [12], and no cross-dataset validation was performed. Transfer across cohorts and collection pipelines therefore remains unmeasured [4]. No participant-stratified, demographic, day-time/nighttime, or subgroup results are available.
5. The five labels are deterministic transformations of a future CGM point, not expert annotations. Their thresholds simplify heterogeneous commercial trend-arrow conventions. The CGM-only model omits meal, insulin, activity, illness, and other covariates.
6. The INT8 calibration set consists of the first 200 validation windows, not a randomized or stratified sample. The observed saturation is therefore a property of this conversion protocol, not of INT8 in general.
7. The phone latency observations use one device, one runtime path, and unequal sample counts without controlled warm-up, thermal, frequency, or

repeated-run analysis. Energy and memory use were not measured.

8. The application replays synthetic or locally extracted historical traces. It does not ingest a live sensor stream and has not undergone prospective, external, usability, or clinical validation.

10 Reproducibility and Ethics

The repository contains the training, conversion, benchmark, Android, and test code. The bundled artifacts were generated from repository commit `361148a` with seed 0. Their SHA-256 digests are `eb96c7e...08e6` (float) and `0dc35387...e29f` (INT8); full hashes are stored beside the assets and asserted by instrumentation tests. Python unit tests cover label boundaries, segment-aware windows, model shape, conversion helpers, and synthetic replay behavior. Android unit tests cover replay, buffering, quantization, latency aggregation, and view-model behavior. A merged-manifest check enforces the absence of Internet permission.

The CGM data are public research data accessed through GlucoBench under the source dataset’s reuse terms [1]. They are not committed to the GLUCOEDGE repository or the distributed Android build. The committed replay trace is synthetic. This work makes no diagnosis, monitoring, treatment, or dosing claim. Its outputs must not be used for medical decisions. The application displays this restriction in the primary interface.

11 Conclusion

GLUCOEDGE is an engineering case study of an offline Android path for a compact five-class CGM trend model, including segment-aware windows, fixed-shape conversion, integer input handling, artifact provenance, cross-runtime parity, and physical-device execution. Float LiteRT preserved the checkpoint metrics and ran in under one millisecond on the tested phone. Static INT8 reduced the already-small artifact by 31.0%, but worsened macro recall, saturated high-valued inputs, and showed no observed latency advantage in this single unequal-sample device run. The broader engineering lesson is that edge-model selection must jointly report class-wise predictive behavior, calibration coverage, artifact size, runtime parity, and real-device measurements. Before any scientific or clinical performance claim, the context-prefix issue must be corrected and repeated seeds, participant-level evaluation, and external validation must be added.

References

- [1] Renat Sergazinov, Elizabeth Chun, Valeriya Rogovchenko, Nathaniel Fernandes, Nicholas Kasman, and Irina Gaynanova. GlucoBench: Curated list of continuous glucose monitoring datasets with prediction benchmarks. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=cUSNs8nGaV>.
- [2] Kui Liu, Linyi Li, Yifei Ma, Jun Jiang, Zhenhua Liu, Zichen Ye, Shuang Liu, Chen Pu, Changsheng Chen, and Yi Wan. Machine learning models for blood glucose level prediction in patients with diabetes mellitus: Systematic review and network meta-analysis. *JMIR Medical Informatics*, 11:e47833, 2023. doi: 10.2196/47833.
- [3] Andrea Facchinetti, Giovanni Sparacino, Emanuele Trifoglio, and Claudio Cobelli. A new index to optimally design and compare continuous glucose monitoring glucose prediction algorithms. *Diabetes Technology & Therapeutics*, 13(2):111–119, 2011. doi: 10.1089/dia.2010.0151.
- [4] Sarala Ghimire, Turgay Celik, Martin Gerdes, and Christian W. Omlin. Deep learning for blood glucose level prediction: How well do models generalize across different data sets? *PLOS ONE*, 19(9): e0310801, 2024. doi: 10.1371/journal.pone.0310801.
- [5] Jeremy Pettus and Steven V. Edelman. Recommendations for using real-time continuous glucose monitoring (rtCGM) data for insulin adjustments in type 1 diabetes. *Journal of Diabetes Science and Technology*, 11(1):138–147, 2017. doi: 10.1177/1932296816663747.
- [6] Melanie Rodacki, Luis Eduardo Calliari, Antonio Carlos Ramalho, et al. Using trend arrows in continuous glucose monitoring systems for insulin adjustment in clinical practice: Brazilian diabetes society position statement. *Diabetology & Metabolic Syndrome*, 13(1):2, 2021. doi: 10.1186/s13098-020-00607-2.
- [7] Kay Henning Brodersen, Cheng Soon Ong, Klaas Enno Stephan, and Joachim M. Buhmann. The balanced accuracy and its posterior distribution. In *20th International Conference on Pattern Recognition*, pages 3121–3124, 2010. doi: 10.1109/ICPR.2010.764.
- [8] Jaques Reifman, Srinivasan Rajaraman, Andrei Gribov, and W. Kenneth Ward. Predictive monitoring for improved management of glucose levels. *Journal of Diabetes Science and Technology*, 1(4):478–486, 2007. doi: 10.1177/193229680700100405.
- [9] Sadegh Mirshekarian, Hui Shen, Razvan Bunescu, and Cindy Marling. LSTMs and neural attention models for blood glucose prediction: Comparative experiments on real and synthetic data. In *41st Annual International Conference of the IEEE Engineering in Medicine and Biology Society*, pages 706–712, 2019. doi: 10.1109/EMBC.2019.8856940.
- [10] John Martinsson, Alexander Schliep, Björn Eliasson, and Olof Mogren. Blood glucose prediction with variance estimation using recurrent neural networks. *Journal of Healthcare Informatics Research*, 4(1):1–18, 2020. doi: 10.1007/s41666-019-00059-y.
- [11] Kezhi Li, John Daniels, Chengyuan Liu, Pau Herero, and Pantelis Georgiou. Convolutional recurrent neural networks for glucose prediction. *IEEE Journal of Biomedical and Health Informatics*, 24(2): 603–613, 2020. doi: 10.1109/JBHI.2019.2908488.
- [12] Ruth S. Weinstock, Stephanie N. DuBose, Richard M. Bergenstal, et al. Risk factors associated with severe hypoglycemia in older adults with type 1 diabetes. *Diabetes Care*, 39(4):603–610, 2016. doi: 10.2337/dc15-1426.
- [13] Christoph Bergmeir, Rob J. Hyndman, and Bonsoo Koo. A note on the validity of cross-validation for evaluating autoregressive time series prediction. *Computational Statistics & Data Analysis*, 120:70–83, 2018. doi: 10.1016/j.csda.2017.11.003.
- [14] Sayash Kapoor and Arvind Narayanan. Leakage and the reproducibility crisis in machine-learning-based science. *Patterns*, 4(9):100804, 2023. doi: 10.1016/j.patter.2023.100804.
- [15] Xavier Bouthillier, Pierre Delaunay, Mirko Bronzi, et al. Accounting for variance in machine learning benchmarks. In *Proceedings of Machine Learning and Systems*, volume 3, pages 747–769, 2021. URL https://proceedings.mlsys.org/paper_files/paper/2021/hash/0184b0cd3cfb185989f858a1d9f5c1eb-Abstract.html.
- [16] Serkan Kiranyaz, Onur Avci, Osama Abdeljaber, Turker Ince, Moncef Gabbouj, and Daniel J. Inman. 1D convolutional neural networks and applications: A survey. *Mechanical Systems and Signal Processing*, 151:107398, 2021. doi: 10.1016/j.ymssp.2020.107398.
- [17] Nicholas D. Lane, Sourav Bhattacharya, Petko Georgiev, Claudio Forlivesi, Lei Jiao, Lorena Qendro, and Fahim Kawsar. DeepX: A software accelerator for low-power deep learning inference on mobile devices. In *2016 15th ACM/IEEE International Conference on Information Processing in Sensor Networks (IPSN)*, pages 1–12, 2016. doi: 10.1109/IPSN.2016.7460664.

- [18] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. AI benchmark: All about deep learning on smartphones in 2019. In *2019 IEEE/CVF International Conference on Computer Vision Workshop (ICCVW)*, pages 3617–3635, 2019. doi: 10.1109/ICCVW.2019.00447.
- [19] Colby Banbury, Vijay Janapa Reddi, Peter Torelli, et al. MLPerf tiny benchmark. *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, 1, 2021. URL https://datasets-benchmarks-proceedings.neurips.cc/paper_files/paper/2021/hash/da4fb5c6e93e74d3df8527599fa62642-Abstract-round1.html.
- [20] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pages 2704–2713, 2018. doi: 10.1109/CVPR.2018.00286.
- [21] PyTorch. PT2E quantization, 2026. URL https://docs.pytorch.org/ao/stable/pt2e_quantization/index.html. Accessed 2026-07-20.
- [22] Google AI Edge. Representative dataset for model quantization. Google AI for Developers documentation, 2026. URL <https://developers.google.com/edge/api/tflite/python/tf/lite/RepresentativeDataset>. Accessed 2026-07-20.
- [23] Ron Banner, Yury Nahshan, and Daniel Soudry. Post training 4-bit quantization of convolutional networks for rapid-deployment. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/c0a62e133894cdce435bcb4a5df1db2d-Abstract.html>.
- [24] Markus Nagel, Rana Ali Amjad, Mart Van Baalen, Christos Louizos, and Tijmen Blankevoort. Up or down? adaptive rounding for post-training quantization. In *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 7197–7206, 2020. URL <https://proceedings.mlr.press/v119/nagel20a.html>.
- [25] Itay Hubara, Yury Nahshan, Yair Hanani, Ron Banner, and Daniel Soudry. Accurate post training quantization with small calibration sets. In *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pages 4466–4475, 2021. URL <https://proceedings.mlr.press/v139/hubara21a.html>.
- [26] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations*, 2015. URL <https://arxiv.org/abs/1412.6980>.
- [27] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://proceedings.neurips.cc/paper/2019/hash/bdbca288fee7f92f2bfa9f7012727740-Abstract.html>.
- [28] Fabian Pedregosa, Gael Varoquaux, Alexandre Gramfort, et al. Scikit-learn: Machine learning in python. *Journal of Machine Learning Research*, 12: 2825–2830, 2011. URL <https://jmlr.org/papers/v12/pedregosa11a.html>.
- [29] Google AI Edge. Run LiteRT compiled model API on Android with Kotlin. Google AI for Developers documentation, 2026. URL https://developers.google.com/edge/litert/next/android_kotlin. Accessed 2026-07-20.
- [30] Google AI Edge. Post-training quantization with LiteRT. Google AI for Developers documentation, 2026. URL https://developers.google.com/edge/litert/conversion/tensorflow/quantization/post_training_quantization. Accessed 2026-07-20.